

Ruprecht-Karls-Universität Heidelberg
Institut für Informatik

Bachelor Thesis
Parallel ASP Planning

Name: Levi Maximilian Klein
Student ID: 4283957
Supervisor: Dr. Gnad
Submission date: 17.02.2026

Ich versichere, dass ich diese Bachelor-Arbeit selbstständig verfasst und nur die angegebenen Quellen und Hilfsmittel verwendet habe.

Heidelberg, dd.mm.yyyy

Zusammenfassung

Dies ist eine Zusammenfassung der Arbeit.

Abstract

This is the abstract.

Contents

List of Figures	1
List of Tables	2
1 Introduction	3
1.1 Motivation	3
1.2 Goal of the Thesis	3
1.3 Structure of the Thesis	3
2 Theory	4
2.1 Definition	4
2.2 ASP	6
2.2.1 encoding for sequential plans	7
2.2.2 encoding for $\forall$ -step plans	8
2.2.3 encoding for $\exists$ -step plans	9
2.2.4 encoding for relaxed $\exists$ -step plans	10
2.3 Proof of Correctness	11
Bibliography	12

List of Figures

List of Tables

1 Introduction

This chapter contains an overview of the topic as well as the goals and contributions of your work.

1.1 Motivation

1.2 Goal of the Thesis

1.3 Structure of the Thesis

Here you describe the structure of the thesis. For example:

In Kapitel [2](#) werden grundlegende Methoden für diese Arbeit vorgestellt.

2 Theory

2.1 Definition

Definition 1 *Multi-valued planning task*. A STRIPS-like multivalued planning task according to (Helmert 2006) is given by a 4 tuple $\langle \mathcal{F}, s_0, s_*, \mathcal{O} \rangle$, where:

- $\mathcal{F}$ is a finite set of state variables, also called **fluents**. Each $x \in \mathcal{F}$ has a finite domain x^d and in every state, x takes exactly one value from its domain.
- s_0 , the **initial state**, is a total function, s.t. $s_0(x) \in x^d$ for each $x \in \mathcal{F}$
- s_* , the **goal conditions**, is a partial function, s.t. $s_*(x) \in x^d$, for each $x \in \text{dom}(s_*)$
- $\mathcal{O}$ is a finite set of operators, also called **actions**. We define an action as $a = \langle a^c, a^e \rangle$, where a^c is the precondition of a and a^e is the postcondition of a . Both a^c and a^e are partial variable assignments over $\mathcal{F}$.

Example 1. Let us illustrate the definitions with a small example. The plan is to leave the house by opening the door, going outside, and finally closing the door.

- $\mathcal{F} = \{\text{door}, \text{location}\}$ with $\text{door}^d = \{\text{open}, \text{closed}\}$, $\text{location}^d = \{\text{inside}, \text{outside}\}$
- Initial state: $s_0 = \{\text{door}=\text{closed}, \text{location}=\text{inside}\}$
- Goal: $s_* = \{\text{door}=\text{closed}, \text{location}=\text{outside}\}$
- Actions: $\mathcal{O} = \{\text{openDoor}, \text{closeDoor}, \text{goOut}\}$, where

$$\begin{aligned} \text{openDoor} &= \langle \{\text{door}=\text{closed}\}, \{\text{door}=\text{open}\} \rangle \\ \text{closeDoor} &= \langle \{\text{door}=\text{open}\}, \{\text{door}=\text{closed}\} \rangle \\ \text{goOut} &= \langle \{\text{location}=\text{inside}, \text{door}=\text{open}\}, \{\text{location}=\text{outside}\} \rangle \end{aligned}$$

Definition 2 *Successor state*. Let s be state and $a \in \mathcal{O}$ an action.

The *successor state* $o(a, s)$, obtained by applying action a in state s , is defined if the precondition of a holds, i.e., $a^c(x) = s(x)$ for all $x \in \text{dom}(a^c)$. Then

$$s'(x) = o(a, s) = \begin{cases} a^e(x) & \text{if } x \in \text{dom}(a^e) \\ s(x) & \text{otherwise.} \end{cases}$$

Remark. Let $A_n = \langle a_1, \dots, a_n \rangle$ be a sequence of actions with length n . We define the application

of A_n to a state s recursively by

$$o(A_n, s) = o(a_n, o(A_{n-1}, s))$$

if $o(A_i, s)$ is defined for all $1 \leq i \leq n$.

Definition 3 Sequential plan. A *sequential plan* is a sequence of actions $A_n = \langle a_1, \dots, a_n \rangle$, s.t. $s' = o(A_n, s_0)$ is defined and $s'(x) = s_*(x)$ for all $x \in \text{dom}(s_*)$.

Example 2. Consider the planning task from the previous example. A sequential plan would be $A = \langle \text{openDoor}, \text{goOut}, \text{closeDoor} \rangle$. Since

$$\begin{aligned} o(A, s_0) &= o(\langle \text{openDoor}, \text{goOut}, \text{closeDoor} \rangle, \{\text{door}=\text{closed}, \text{location}=\text{inside}\}) \\ &= o(\langle \text{goOut}, \text{closeDoor} \rangle, \{\text{door}=\text{open}, \text{location}=\text{inside}\}) \\ &= o(\text{closeDoor}, \{\text{door}=\text{open}, \text{location}=\text{outside}\}) \\ &= \{\text{door}=\text{closed}, \text{location}=\text{outside}\} = s_* \end{aligned}$$

To get to parallel actions and plans, we need the definition of confluent actions.

Definition 4 confluent. Let $A = \{a_1, \dots, a_n\} \subseteq \mathcal{O}$ be a set of actions.

A is *confluent* $:\Leftrightarrow a_i^e(x) = a_j^e(x)$ for all $1 \leq i < j \leq n$ and each $x \in \text{dom}(a_i^e) \cap \text{dom}(a_j^e)$

Remark. A is confluent $\Leftrightarrow B$ is confluent for all $B \subseteq A$

Example 3. In our example $\{\text{openDoor}, \text{goOut}\}$ and $\{\text{closeDoor}, \text{goOut}\}$ are confluent sets.

We extend our example with the fluents *light* and *key* with $\text{light}^d = \text{key}^d = \{0, 1\}$ and two actions *lightsOut* and *getKey*:

$$\begin{aligned} \text{lightsOut} &= \langle \{\text{location}=\text{inside}, \text{light}=1\}, \{\text{light}=0\} \rangle \\ \text{getKey} &= \langle \{\text{location}=\text{inside}, \text{key}=0\}, \{\text{key}=1\} \rangle \end{aligned}$$

Additionally, we modify the precondition: $\text{openDoor}^c = \{\text{door}=\text{closed}, \text{key}=1\}$. Our new goal is $s_* = \{\text{location}=\text{outside}, \text{door}=\text{closed}, \text{light}=0\}$.

Now for example $\{\text{getKey}, \text{openDoor}, \text{goOut}, \text{lightsOut}\}$ and $\{\text{getKey}, \text{closeDoor}, \text{goOut}, \text{lightsOut}\}$ are also confluent sets.

Definition 5 Parallel representations. Let s be a state and $A = \{a_1, \dots, a_n\}$ be a confluent set of actions. Then A is

- (i) $\forall$ -step serializable in $s : \Leftrightarrow o(\langle a_{\pi(1)}, \dots, a_{\pi(n)} \rangle, s)$ is defined for all permutations π
- (ii) $\exists$ -step serializable in $s : \Leftrightarrow o(\langle a_{\pi(1)}, \dots, a_{\pi(n)} \rangle, s)$ is defined for some permutation π
and $a^c(x) = s(x)$ for each $a \in A$ and $x \in \text{dom}(a^c)$
- (iii) relaxed $\exists$ -step serializable in $s : \Leftrightarrow o(\langle a_{\pi(1)}, \dots, a_{\pi(n)} \rangle, s)$ is defined for some permutation π

Remark. A is $\forall$ -step serializable $\Rightarrow A$ is $\exists$ -step serializable $\Rightarrow A$ is relaxed $\exists$ -step serializable.

Definition 6 Parallel Plans. Let $A = \langle A_1, \dots, A_n \rangle$ be a sequence of confluent sets of actions. A is a $\forall$ -step, $\exists$ -step or relaxed $\exists$ -step plan iff

- $s_n(x) = s_*(x)$ for each $x \in \text{dom}(s_*)$
- A_i is $\forall$ -step, $\exists$ -step or relaxed $\exists$ -step serializable in s_{i-1} for $1 \leq i \leq n$, where

$$s_i(x) = \begin{cases} a^e(x) & \text{if } x \in \text{dom}(a^e) \\ s_{i-1}(x) & \text{else} \end{cases} \quad \text{for each } a \in A_i$$

Example 4. In our modified example we have:

- $\forall$ -step plan: $\langle \{getKey, lightsOut\}, \{openDoor\}, \{goOut\}, \{closeDoor\} \rangle$
- $\exists$ -step plan: $\langle \{getKey, lightsOut\}, \{openDoor\}, \{goOut, closeDoor\} \rangle$
- relaxed $\exists$ -step plan: $\langle \{getKey, lightsOut, openDoor, goOut\}, \{closeDoor\} \rangle$

2.2 ASP

Idee: Alles mit den mathematische Formeln von oben beschreiben. In ASP we represent planning tasks by facts. A solution of an ASP encoding is called stable model.

```

1  fluent(door; loc; light; key).
2  value(door, 0).  value(door, 1).
3  value(loc, in).  value(loc, out).
4  value(light, 0). value(light, 1).
5  value(key, 0).   value(key, 1).
6
7  init(door, 0).  init(loc, in).  init(light, 1).  init(key, 0).
8  goal(door, 0).  goal(loc, out).  goal(light, 0).
9
10 action(openDoor; closeDoor; goOut; lightOut; getKey).
11 prec(openDoor, door, 0).  prec(openDoor, key, 1).  post(openDoor, door, 1).
12 prec(closeDoor, door, 1).  post(closeDoor, door, 0).
13 prec(goOut, loc, in).  prec(goOut, door, 1).  post(goOut, loc, out).
14 prec(lightOut, loc, in).  prec(lightOut, light, 1).  post(lightOut, light, 0).
15 prec(getKey, loc, in).  prec(getKey, key, 0).  post(getKey, key, 1).

```

Listing 1: ASP encoding of the example planning task

Combining these facts with the right encodings, stable models corresponds to the *sequential*, $\forall$ -step, $\exists$ -step or relaxed $\exists$ -step plan.

```

1  #include <incmode>.
2  holds(X, V, 0) :- init(X, V).
3  #program check(t).
4  :- query(t), goal(X, V), not holds(X, V, t).
5  #program step(t).
6  {holds(X, V, t) : value(X, V)} = 1 :- fluent(X).
7  {occurs(A, t)} :- action(A).
8  :- occurs(A, t), post(A, X, V), not holds(X, V, t).
9  :- holds(X, V, t), not holds(X, V, t-1), not occurs(A, t) : post(A, X, V).

```

Listing 2: Core for sequential and parallel encodings for planning

All planning algorithms share a common incremental encoding using the incmode for clingo as seen in Listing 2 (Gebser et al. 2014). The program is divided into three subprograms: *base*, *step(t)* and *check(t)*. The *base* subprogram contains the static part of the encoding, which is only grounded once. Line 2 defines the initial state s_0 , where the predicate $holds(X, V, t)$ is used to represent fluents $X \in \mathcal{F}$ that have value $V \in dom(X)$ at timestep t .

check(t) is a parametrized subprogram starting at $t = 0$. The integrity constraint in Line 4 checks if $s_*(X) = s_t(X) \forall X \in dom(s_*)$. *Query(t)* is true only for the latest integer t .

step(t) is grounded after check fails for each timestep t starting at $t = 1$. Note that postconditions of applied actions in timestep t are also set in t and the preconditions have to hold in $t - 1$. Lines 6 defines holds for times step t , giving fluent X one value V from its domain and *occurs(A, t)* in Line 7 are all possible actions in timestep t obeying rules 8 and 9. Line 8 ensures that the postcondition of an action is applied and also assert the confluence of any set of action applied in parallel since if $V = a^e(X) \neq \tilde{a}(X) = \tilde{V}$ either $holds(X, V, t)$ or $holds(X, \tilde{V}, t)$ would be false. Line 9 makes sure that fluents not affected by any action keep their value from the previous timestep. So far, the common core handles the transitions from one state to the next in regards of the postcondition of applied actions.

2.2.1 encoding for sequential plans

```

1  :- occurs(A, t), prec(A, X, V), not holds(X, V, t-1).
2  :- #count{A : occurs(A, t)} > 1.

```

Listing 3: Extension for sequential plans

Starting with the extension for sequential plans as seen in Listing 3, Line 2 ensures that the precondition of an applied action is met in $t - 1$ and Line 3 only allows one action to occur per timestep.

Example 5. Running this with our example, we get three possible sequential plans.

```

Answer: 1
occurs(getKey,1) occurs(lightOut,2) occurs(openDoor,3) occurs(goOut,4) occurs(closeDoor,5)
Answer: 2
occurs(getKey,1) occurs(openDoor,2) occurs(lightOut,3) occurs(goOut,4) occurs(closeDoor,5)
Answer: 3
occurs(lightOut,1) occurs(getKey,2) occurs(openDoor,3) occurs(goOut,4) occurs(closeDoor,5)

```

Listing 4: Output from clingo for sequential encoding

2.2.2 encoding for $\forall$ -step plans

```

1 :- occurs(A, t), prec(A, X, V), not holds(X, V, t-1).
2 :- occurs(A, t), prec(A, X, V), not post(A, X, _), not holds(X, V, t).
3 single(X, t) :- occurs(A, t), prec(A, X, V1), post(A, X, V2), V1 != V2.
4 :- single(X, t), #count{A : occurs(A, t), post(A, X, V)} > 1.

```

Listing 5: Extension for $\forall$ -step plans

Now for the extension for $\forall$ -step plans as seen in Listing 5, Line 1 is the same as in the sequential encoding. As mentioned before, the integrity constraint in Line 8 of the common core already ensures confluence of applied actions. Since the sequence of the actions has to be valid for all permutations, we have to make sure that no action invalidates the precondition of any other action. Let a be an action applied in parallel and $X \in \text{dom}(a^c)$. There are two cases to consider:

1. the fluent X remains unchanged, i.e. $X \in \text{dom}(a^c) \cap \text{dom}(a^e)$ and $a^c(X) = a^e(X)$ or $X \in \text{dom}(a^c)$ and $X \notin \text{dom}(a^e)$
2. the postcondition of a invalidates its precondition, i.e. $X \in \text{dom}(a^e) \cap a^c(X) \neq a^e(X)$ for an action a

Line 2 handles the first case, while the second case is a little trickier. If there is such an action, there can not be another action $\tilde{a}$ with $X \in \text{dom}(\tilde{a}^e)$, since if $a^e(X) = \tilde{a}^e(X)$, the precondition of a is invalidated and if $a^e(X) \neq \tilde{a}^e(X)$, the confluence is undermined. For this, we define these actions described in (ii) as $\text{single}(X, t)$ in Line 3 and the integrity constraint in Line 4 ensures

that there is at most one such action for each fluent X in timestep t .

Example 6. Running this with our example, we get two possible $\forall$ -step plans.

```

Answer: 1
occurs(getKey,1) occurs(openDoor,2) occurs(lightOut,2) occurs(goOut,3) occurs(closeDoor,4)
Answer: 2
occurs(lightOut,1) occurs(getKey,1) occurs(openDoor,2) occurs(goOut,3) occurs(closeDoor,4)

```

Listing 6: Output from clingo for $\forall$ -step encoding

2.2.3 encoding for $\exists$ -step plans

```

1 :- occurs(A, t) , prec(A, X, V), not holds(X, V, t-1).
2 apply(A1, t) :- action(A1),
3     ready(A2, t) : post(A1, X, V1), prec(A2, X, V2), A1 != A2, V1 != V2.
4 ready(A, t) :- action(A), not occurs(A, t).
5 ready(A, t) :- apply(A, t).
6 :- action(A) , not ready(A, t).

```

Listing 7: Extension for $\exists$ -step plans

```

1 :- occurs(A, t), prec(A, X, V), not holds(X, V, t-1).
2 #edge((A1, t), (A2, t)): occurs(A1, t),
3     post(A1, X, V1), prec(A2, X, V2), A1 != A2, V1 != V2.

```

Listing 8: Alternative Extension for $\exists$ -step plans

There are two different ways to encode the extension for $\exists$ -step plans as seen in Listing 7 and Listing 8. As before, Line 1 in Listing 7 and 8 is the same as in the sequential and $\forall$ -step encoding. To ensure that there is at least one valid permutation, either an action a_1 that invalidates the precondition of another action a_2 has to be applied after a_2 or action a_2 does not occur in this timestep. For this, all actions has to be marked as *ready* (Line 6). An action is *ready* if it either does not occur in this timestep (Line 4) or if it is captured by *apply* (Line 2). An action can be safely applied, if all other actions whose preconditions it invalidates are

ready, i.e. are either applied beforehand or not occurring in this timestep. Line 6 also ensures that actions does not circularly invalidates each other.

Example 7. Running this with our example, we get three possible $\exists$ -step plans.

```

Answer: 1
occurs(getKey,1) occurs(openDoor,2) occurs(closeDoor,3) occurs(goOut,3) occurs(lightOut,3)
Answer: 2
occurs(getKey,1) occurs(openDoor,2) occurs(lightOut,2) occurs(closeDoor,3) occurs(goOut,3)
Answer: 3
occurs(lightOut,1) occurs(getKey,1) occurs(openDoor,2) occurs(closeDoor,3) occurs(goOut,3)

```

Listing 9: Output from clingo for $\exists$ -step encoding

2.2.4 encoding for relaxed $\exists$ -step plans

```

1 reach(X, V, t) :- holds(X, V, t - 1).
2 reach(X, V, t) :- occurs(A, t), apply(A, t), post(A, X, V).
3 apply(A1, t) :- action(A1), reach(X, V, t) : prec(A1, X, V);
4     ready(A2, t) : post(A1, X, V1), prec(A2, X, V2), A1 != A2 , V1 != V2.
5 ready(A, t) :- action(A), not occurs(A, t).
6 ready(A, t) :- apply(A, t).
7 :- action(A), not ready(A, t).

```

Listing 10: Extension for relaxed $\exists$ -step plans

Finally, the extension for relaxed $\exists$ -step plans as seen in Listing 10 is quite similar to the $\exists$ -step encoding. The difference between those two is that the precondition of all actions do not have to hold simultaneous but they can "help" each other out. This is done by defining the predicate `reach/3` in Line 1 and 2 as all fluents that can be reached by the step before or applied action in this timestep. An action can then be safely applied, if its precondition is reachable and as before, all actions that it invalidates are ready (Line 4). Again Line 7 ensures that applied action do not circularly invalidate each other.

Example 8. Running this with our example, we get three possible relaxed $\exists$ -step plans.

```

Answer: 1
occurs(openDoor,1) occurs(getKey,1) occurs(closeDoor,2) occurs(goOut,2) occurs(lightOut,2)
Answer: 2
occurs(openDoor,1) occurs(lightOut,1) occurs(getKey,1) occurs(closeDoor,2) occurs(goOut,2)
Answer: 3
occurs(openDoor,1) occurs(goOut,1) occurs(lightOut,1) occurs(getKey,1) occurs(closeDoor,2)

```

Listing 11: Output from clingo for relaxed $\exists$ -step encoding

2.3 Proof of Correctness

In order to prove the correctness of the presented encodings, we define the following:

- B is the rule in Line 1 of Listing 3.
- $Q(t)$ is the integrity constraint in the check program directive (Line 5) of Listing 2.
- $S(t)$ are the rules and integrity constraint in the step program directive (Lines 6-9) of Listing 2.
- $S^\square(t)$ is the extension for the specific plans, where $\square = s$ for Listing 3, $\square = \forall$ for Listing 5, $\square = \exists$ for Listing 7, and $\square = R$ for Listing 10.

For the next theorems, let I be the set of facts representing a planning task $\langle \mathcal{F}, s_0, s_*, \mathcal{O} \rangle$, $\langle a_1, \dots, a_n \rangle$ be a sequence of actions and $\langle A_1, \dots, A_m \rangle$ be a sequence of sets of actions.

Theorem 1 (Correctness of sequential plans).

$\langle a_1, \dots, a_n \rangle$ is a sequential plan iff

$$I \cup B \cup Q(0) \cup \bigcup_{t=1}^n (Q(t) \cup S(t) \cup S^s(t)) \cup \{query(n).\}$$

has a stable model M , s.t. $\{\langle a, t \rangle | occurs(a, t) \in M\} = \{\langle a_t, t \rangle | 1 \leq t \leq n\}$.

Proof. Let $P^s(n) = I \cup B \cup Q(0) \cup \bigcup_{t=1}^n (Q(t) \cup S(t) \cup S^s(t)) \cup \{query(n).\}$.

$(\Rightarrow)$ $\langle a_1, \dots, a_n \rangle$ is a sequential plan $\Rightarrow$ any stable model M of $P^s(n)$ s.t. $\{\langle a, t \rangle | occurs(a, t) \in M\} = \{\langle a_t, t \rangle | 1 \leq t \leq n\}$ the rule in B ($holds(X, V, 0) = s_0 = (X)$), the choice rule in $S(t)$ (for given X and t , there is only one V for $holds(X, V, t)$) and both integrity constraints in $S(i)$ ($holds(X, V, t) = a^e(X)$ if $X \in dom(a^e)$, $s_{t-1}(X)$ otherwise) make sure that $holds(X, V, 0) = s_0(X)$ and $holds(X, V, t) = o(a_t, s_{t-1})(X)$, i.e.

$$\{\langle X, V, t \rangle | holds(X, V, t) \in M\} = \{\langle X, V, t \rangle | X \in \mathcal{F}, v = o(\langle a_1, \dots, a_t \rangle, s_0)\}.$$

□

Bibliography

- Helmert, M. (2006). “The Fast Downward Planning System”. In: *Journal of Artificial Intelligence Research* 26, pp. 191–246.
- Gebser, Martin et al. (May 2014). *Clingo = ASP + control: Preliminary report*. Tech. rep. 1405.3694. arXiv. URL: <https://arxiv.org/abs/1405.3694v1>.